

CoC 7e LLM Keeper — v2.7.6.1 “The Truth Firewall” Patch Pack

Roadmap phase: Phase 0 — Truth Firewall

Baseline: v2.7.6 “The Spoiler Channel”

Status: Implementation progress pack, ready to merge into the editable source tree

Scope completed in this pack

This patch begins the first roadmap phase and implements the core Phase 0 safety work:

- New `src/state_validator.py`
- Model-produced `state_delta` is validated before touching world state
- Engine-owned character fields are blocked from direct LLM mutation
- Scene transitions must follow the actual location graph
- Front clocks must reference existing fronts and clamp to `0..max`
- Unknown top-level `state_delta` fields are rejected
- Safe proposal fields remain available
- Persistent per-character weapon instances are added
- Unequip/equip no longer restores ammunition from the catalog
- Save/load preserves weapon ammunition and condition
- Active-character prompt serialization now includes carried inventory
- Phase 0 regression tests and release notes are included

The uploaded project was available only as the complete-source PDF, not as an editable source tree. This pack therefore contains exact replacement code and targeted merge blocks for the editable project. The standalone validator and weapon-persistence logic have both passed local regression tests.

Merge order

Apply in this order:

1. Add `src/state_validator.py`.
2. Replace or merge `src/character.py`.
3. Apply the `src/keeper.py` edits.
4. Update `config/system-prompt.txt`.
5. Update `src/__init__.py`.
6. Add the `test_engine.py` regression block.
7. Add `docs/v2.7.6.1-release-notes.md`.
8. Run the offline suites.

Expected commands:

```
python test_engine.py
python test_charcreate.py
python -m src.main --mock
```

1. New file: `src/state_validator.py`

```
"""State-delta validation for the CoC 7e LLM Keeper.

v2.7.6.1 — the Truth Firewall.

The LLM may narrate and propose consequences, but it may not directly assign
canonical mechanical state. This module accepts a model-produced state_delta,
returns the subset that is safe for the engine to apply, and records every
rejected write for debugging and regression tests.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Dict, List, Mapping, Optional, Set

# Canonical state owned by deterministic systems. The model may describe
# changes to these things, but it may not assign them directly.
ENGINE_OWNED_CHARACTER_FIELDS: Set[str] = {
    "id",
    "name",
    "char_type",
    "owner",
    "STR",
    "CON",
    "SIZ",
    "DEX",
    "APP",
    "INT",
    "POW",
    "EDU",
    "hp",
    "max_hp",
    "san",
    "max_san",
    "mp",
    "luck",
    "build",
    "damage_bonus",
    "move",
    "skills",
    "checked_skills",
    "major_wound",
    "dying",
    "unconscious",
    "temporarily_insane",
    "indefinitely_insane",
    "cthulhu_mythos",
    "san_loss_today",
}
```

```

    "weapon",
    "weapon_instances",
    "armor",
    "scars",
    "phobias",
    "manias",
    "key_items",
    "inventory",
    "location",
}

# Narrative/adjacent fields the model may still update during the transition
# period. Phase 8 moves these into explicit proposal fields as well.
MODEL_SAFE_CHARACTER_FIELDS: Set[str] = {
    "position",
    "declared_action",
    "personal_log",
    "extra",
}

# State-delta sections that contain proposals rather than direct mutations.
PROPOSAL_TOP_LEVEL_FIELDS: Set[str] = {
    "proposed_facts",
    "proposed_consequences",
    "npc_reactions",
    "sound_events",
}

def _short(value: Any, limit: int = 120) -> str:
    text = repr(value)
    if len(text) > limit:
        return text[: limit - 3] + "..."
    return text

@dataclass(frozen=True)
class StateRejection:
    """One attempted model write that the engine refused."""

    path: str
    value: Any
    reason: str

    def format(self) -> str:
        return f"{self.path}: {self.reason} (value={_short(self.value)})"

@dataclass
class StateValidationReport:
    """Cleaned delta plus an audit trail of rejected writes."""

```

```

delta: Dict[str, Any]
rejected: List[StateRejection] = field(default_factory=list)

@property
def ok(self) -> bool:
    return not self.rejected

class StateDeltaValidator:
    """Validate model-produced state deltas before they touch world state."""

    def validate(
        self,
        delta: Optional[Mapping[str, Any]],
        *,
        characters: Mapping[str, Any],
        fronts: Mapping[str, Any],
        locations: Mapping[str, Any],
    ) -> StateValidationReport:
        if not isinstance(delta, Mapping):
            return StateValidationReport(
                {},
                [StateRejection("state_delta", delta, "state_delta must be an object")],
            )

        clean: Dict[str, Any] = {}
        rejected: List[StateRejection] = []

        for key, value in delta.items():
            if key == "characters":
                accepted, errors = self._validate_characters(value, characters)
            elif key == "fronts":
                accepted, errors = self._validate_fronts(value, fronts)
            elif key == "plot_points":
                accepted, errors = self._validate_plot_points(value)
            elif key == "scene_transitions":
                accepted, errors = self._validate_scene_transitions(
                    value, characters=characters, locations=locations
                )
            elif key in PROPOSAL_TOP_LEVEL_FIELDS:
                accepted, errors = self._validate_proposal_field(key, value, characters, locations)
            else:
                accepted, errors = None, [
                    StateRejection(
                        f"state_delta.{key}",
                        value,
                        "top-level field is not approved for model writes",
                    )
                ]

```

```

        rejected.extend(errors)
        if accepted not in (None, {}, []):
            clean[key] = accepted

    return StateValidationReport(clean, rejected)

# ----- characters
def _validate_characters(
    self, value: Any, characters: Mapping[str, Any]
) -> tuple[Dict[str, Dict[str, Any]], List[StateRejection]]:
    if not isinstance(value, Mapping):
        return {}, [StateRejection("state_delta.characters", value, "must be an object")]

    accepted: Dict[str, Dict[str, Any]] = {}
    rejected: List[StateRejection] = []

    for char_id, changes in value.items():
        path = f"state_delta.characters.{char_id}"
        if char_id not in characters:
            rejected.append(StateRejection(path, changes, "unknown character"))
            continue
        if not isinstance(changes, Mapping):
            rejected.append(StateRejection(path, changes, "character changes must be an object"))
            continue

        safe_changes: Dict[str, Any] = {}
        for field_name, field_value in changes.items():
            field_path = f"{path}.{field_name}"
            if field_name in ENGINE_OWNED_CHARACTER_FIELDS:
                rejected.append(
                    StateRejection(
                        field_path,
                        field_value,
                        "engine-owned field; use an engine event or proposal",
                    )
                )
            elif field_name in MODEL_SAFE_CHARACTER_FIELDS:
                safe_changes[field_name] = field_value
            else:
                rejected.append(
                    StateRejection(
                        field_path,
                        field_value,
                        "field is not approved for direct model writes",
                    )
                )

        if safe_changes:
            accepted[char_id] = safe_changes

    return accepted, rejected

```

```

# ----- fronts
def _validate_fronts(
    self, value: Any, fronts: Mapping[str, Any]
) -> tuple[Dict[str, int], List[StateRejection]]:
    if not isinstance(value, Mapping):
        return {}, [StateRejection("state_delta.fronts", value, "must be an object")]

    accepted: Dict[str, int] = {}
    rejected: List[StateRejection] = []

    for front_id, raw_clock in value.items():
        path = f"state_delta.fronts.{front_id}"
        front = fronts.get(front_id)
        if front is None:
            rejected.append(StateRejection(path, raw_clock, "unknown front"))
            continue
        if not isinstance(raw_clock, (int, float)) or isinstance(raw_clock, bool):
            rejected.append(StateRejection(path, raw_clock, "front clock must be numeric"))
            continue

        maximum = front.get("max", raw_clock) if isinstance(front, Mapping) else raw_clock
        try:
            maximum = int(maximum)
        except (TypeError, ValueError):
            maximum = int(raw_clock)
        accepted[front_id] = max(0, min(int(raw_clock), maximum))

    return accepted, rejected

# ----- plot points
def _validate_plot_points(self, value: Any) -> tuple[List[str], List[StateRejection]]:
    if not isinstance(value, list):
        return [], [StateRejection("state_delta.plot_points", value, "must be a list")]

    accepted: List[str] = []
    rejected: List[StateRejection] = []
    for i, point in enumerate(value):
        if isinstance(point, str) and point.strip():
            accepted.append(point.strip())
        else:
            rejected.append(
                StateRejection(
                    f"state_delta.plot_points[{i}]",
                    point,
                    "plot points must be non-empty strings",
                )
            )
    return accepted, rejected

# ----- scene transitions

```

```

def _validate_scene_transitions(
    self,
    value: Any,
    *,
    characters: Mapping[str, Any],
    locations: Mapping[str, Any],
) -> tuple[List[Dict[str, str]], List[StateRejection]]:
    if not isinstance(value, list):
        return [], [
            StateRejection("state_delta.scene_transitions", value, "must be a list")
        ]

    accepted: List[Dict[str, str]] = []
    rejected: List[StateRejection] = []

    for i, transition in enumerate(value):
        path = f"state_delta.scene_transitions[{i}]"
        if not isinstance(transition, Mapping):
            rejected.append(StateRejection(path, transition, "transition must be an object"))
            continue

        clean_transition: Dict[str, str] = {}
        for char_id, destination in transition.items():
            char_path = f"{path}.{char_id}"
            char = characters.get(char_id)
            if char is None:
                rejected.append(StateRejection(char_path, destination, "unknown character"))
                continue
            if not isinstance(destination, str) or destination not in locations:
                rejected.append(StateRejection(char_path, destination, "unknown destination"))
                continue

            current = getattr(char, "location", None)
            current_location = locations.get(current)
            if destination == current:
                clean_transition[char_id] = destination
                continue

            connections = getattr(current_location, "connections", {}) or {}
            if current_location is None or destination not in connections:
                rejected.append(
                    StateRejection(
                        char_path,
                        destination,
                        f"destination is not connected to current location {current!r}",
                    )
                )
                continue

            clean_transition[char_id] = destination

```

```

        if clean_transition:
            accepted.append(clean_transition)

    return accepted, rejected

# ----- proposals
def _validate_proposal_field(
    self,
    key: str,
    value: Any,
    characters: Mapping[str, Any],
    locations: Mapping[str, Any],
) -> tuple[Any, List[StateRejection]]:
    """Validate proposal payloads without treating them as state writes."""
    if key in ("proposed_facts", "proposed_consequences", "npc_reactions"):
        if not isinstance(value, list):
            return None, [
                StateRejection(f"state_delta.{key}", value, "proposal field must be a list")
            ]
        return list(value), []

    if key == "sound_events":
        if not isinstance(value, list):
            return None, [
                StateRejection("state_delta.sound_events", value, "sound events must be a list")
            ]

    accepted = []
    rejected: List[StateRejection] = []
    for i, event in enumerate(value):
        path = f"state_delta.sound_events[{i}]"
        if not isinstance(event, Mapping):
            rejected.append(StateRejection(path, event, "sound event must be an object"))
            continue
        location = event.get("location")
        noise = event.get("noise")
        heard_by = event.get("heard_by", [])
        if location is not None and location not in locations:
            rejected.append(StateRejection(f"{path}.location", location, "unknown location"))
            continue
        if not isinstance(noise, int) or isinstance(noise, bool) or not 1 <= noise <= 5:
            rejected.append(
                StateRejection(f"{path}.noise", noise, "noise must be an integer from 1 to 5")
            )
            continue
        if not isinstance(heard_by, list):
            rejected.append(StateRejection(f"{path}.heard_by", heard_by, "heard_by must be a
↪ list"))
            continue
        clean_event = dict(event)
        clean_event["heard_by"] = [cid for cid in heard_by if cid in characters]

```



```
        accepted.append(clean_event)
    return accepted, rejected

    return None, [StateRejection(f"state_delta.{key}", value, "unsupported proposal field")]
```

2. Replace or merge: **src/character.py**

If your editable `src/character.py` differs from the v2.7.6 PDF bundle, merge the sections marked v2.7.6.1 instead of blindly replacing the file. The complete intended file is below.

```
"""Character and Weapon models for CoC 7e.

v2.7.6.1 — Truth Firewall support:
- Adds persistent per-character weapon instances so unequip/equip, save/load,
  and combat all refer to the same physical weapon. This is the temporary
  bridge until the full item registry lands in v2.8.0.
"""

from dataclasses import asdict, dataclass, field
from typing import Dict, List, Optional

@dataclass
class Weapon:
    name: str
    damage: str
    base_range: int          # yards; 0 = melee only
    rof: int = 1
    ammo: int = 6
    malfunction: int = 96
    is_shotgun: bool = False
    is_short_barrel: bool = False

    def get_range_band(self, distance: float, dex: int) -> str:
        point_blank = dex / 5 if (self.is_shotgun or self.is_short_barrel) else 2
        if distance <= point_blank:
            return "point_blank"
        if self.base_range <= 0:
            return "out_of_range"
        if distance <= self.base_range:
            return "regular"
        if distance <= self.base_range * 2:
            return "long"
        return "extreme"

    def get_skill_target(self, base_skill: int, band: str) -> int:
        # CoC 7e: regular range = full skill, long = half (Hard), extreme = fifth (Extreme)
        if band in ("point_blank", "regular"):
            return base_skill
```

```

        if band == "long":
            return base_skill // 2
        return base_skill // 5

@dataclass
class Character:
    id: str
    name: str
    char_type: str # "player" | "npc"
    owner: Optional[str] = None
    STR: int = 50
    CON: int = 50
    SIZ: int = 50
    DEX: int = 50
    APP: int = 50
    INT: int = 50
    POW: int = 50
    EDU: int = 50
    hp: Optional[int] = None # None -> derive (max_hp)
    max_hp: Optional[int] = None # None -> derive (CON+SIZ)/10
    san: Optional[int] = None # None -> derive (POW)
    max_san: Optional[int] = None # None -> derive 99 - mythos
    mp: Optional[int] = None # None -> derive POW/5
    luck: int = 50
    build: int = 0
    damage_bonus: str = "0"
    move: int = 8
    skills: Dict[str, int] = field(default_factory=dict)
    checked_skills: List[str] = field(default_factory=list)
    major_wound: bool = False
    dying: bool = False
    unconscious: bool = False
    temporarily_insane: bool = False
    indefinitely_insane: bool = False
    cthulhu_mythos: int = 0
    san_loss_today: int = 0
    weapon: Optional[Weapon] = None
    # v2.7.6.1: physical weapon instances owned by this character, keyed by
    # current display name. The v2.8.0 item registry replaces this bridge.
    weapon_instances: Dict[str, Weapon] = field(default_factory=dict)
    armor: int = 0
    scars: List[str] = field(default_factory=list)
    phobias: List[str] = field(default_factory=list)
    manias: List[str] = field(default_factory=list)
    key_items: List[str] = field(default_factory=list)
    # v2.7.3: what they carry (display names of weapons). The equipped piece
    # is `weapon`; the pack is `inventory`. Managed by meta-commands
    # (inventory / equip / unequip), never by prose.
    inventory: List[str] = field(default_factory=list)
    location: str = "unknown"

```

```

position: str = "close"                # close/near/far/elevated/behind_cover
declared_action: str = ""
personal_log: List[str] = field(default_factory=list)
extra: Dict = field(default_factory=dict)  # attitude, notes, anything scenario-specific

def __post_init__(self):
    if self.max_hp is None:
        self.max_hp = (self.CON + self.SIZ) // 10
    if self.hp is None:
        self.hp = self.max_hp
    if self.san is None:
        self.san = self.POW
    if self.max_san is None:
        self.max_san = 99 - self.cthulhu_mythos
    if self.mp is None:
        self.mp = self.POW // 5

    str_siz = self.STR + self.SIZ
    if str_siz <= 64:
        self.build, self.damage_bonus = -2, "-2"
    elif str_siz <= 84:
        self.build, self.damage_bonus = -1, "-1"
    elif str_siz <= 124:
        self.build, self.damage_bonus = 0, "0"
    elif str_siz <= 164:
        self.build, self.damage_bonus = 1, "+1D4"
    else:
        self.build, self.damage_bonus = 2, "+1D6"

    if self.DEX < self.SIZ and self.STR < self.SIZ:
        self.move = 7
    elif self.DEX > self.SIZ and self.STR > self.SIZ:
        self.move = 9
    else:
        self.move = 8

    # The piece in hand is, by definition, something they carry. This
    # also migrates pre-v2.7.3 saves: an equipped weapon with an empty
    # inventory seeds itself.
    if not self.inventory and self.weapon:
        self.inventory = [self.weapon.name]

    # v2.7.6.1: seed the persistent instance store from the equipped
    # weapon. Old saves therefore migrate on first load.
    if self.weapon and self.weapon.name not in self.weapon_instances:
        self.weapon_instances[self.weapon.name] = self.weapon

    # ---- status -----
def get_condition(self) -> str:
    if self.dying:
        return "dying"

```

```

    if self.unconscious:
        return "unconscious"
    if self.major_wound:
        return "major_wound"
    if self.hp is not None and self.hp <= self.max_hp // 2:
        return "wounded"
    return "healthy"

def take_damage(self, damage: int):
    net = max(0, damage - self.armor)
    if net >= self.max_hp // 2:
        self.major_wound = True
    self.hp = max(0, self.hp - net)
    if self.hp == 0:
        if self.major_wound:
            self.dying = True
        else:
            self.unconscious = True
    return net

# ---- prompt serialization -----
def to_active_format(self) -> dict:
    return {
        "id": self.id,
        "name": self.name,
        "type": self.char_type,
        "hp": self.hp,
        "max_hp": self.max_hp,
        "san": self.san,
        "condition": self.get_condition(),
        "key_skills": dict(list(self.skills.items())[:5]),
        "weapon": self.weapon.name if self.weapon else None,
        "inventory": list(self.inventory),
        "position": self.position,
        "action": self.declared_action,
    }

def to_summary_format(self) -> dict:
    return {
        "id": self.id,
        "name": self.name,
        "condition": self.get_condition(),
        "location": self.location,
        "status": self.declared_action or "Idle",
    }

# ---- persistence -----
def to_dict(self) -> dict:
    d = asdict(self)
    d["weapon"] = asdict(self.weapon) if self.weapon else None
    d["weapon_instances"] = {

```

```

        name: asdict(weapon) for name, weapon in self.weapon_instances.items()
    }
    return d

    @classmethod
    def from_dict(cls, d: dict) -> "Character":
        d = dict(d)
        weapon = d.get("weapon")
        if isinstance(weapon, dict):
            known = set(Weapon.__dataclass_fields__)
            d["weapon"] = Weapon(**{k: v for k, v in weapon.items() if k in known})

        instances = d.get("weapon_instances") or {}
        if isinstance(instances, dict):
            known = set(Weapon.__dataclass_fields__)
            d["weapon_instances"] = {
                name: Weapon(**{k: v for k, v in value.items() if k in known})
                for name, value in instances.items()
                if isinstance(value, dict)
            }
        else:
            d["weapon_instances"] = {}

        known = set(cls.__dataclass_fields__)
        return cls(**{k: v for k, v in d.items() if k in known})

```

3. Targeted edits: **src/keeper.py**

3.1 Add the validator import

Find the existing imports near the top:

```

from src.charcreate import skill_base
from src import state as state_mod

```

Replace with:

```

from src.charcreate import skill_base
from src import state as state_mod
from src.state_validator import (
    ENGINE_OWNED_CHARACTER_FIELDS,
    StateDeltaValidator,
)

```

3.2 Initialize the validator

Inside `CoCKeeper.__init__`, after the existing engine setup lines:

```

self.combat = CombatEngine(self.spatial, self.dice)
self.sanity = SanityEngine(self.dice, self.combat, config.get("sanity", {}))

```

```
self.chronicle = build_chronicle(config)
```

Add:

```
# v2.7.6.1: every model-produced state delta passes through the Truth
# Firewall before it can touch campaign state.
self.state_validator = StateDeltaValidator()
```

3.3 Replace the unequip branch in `_meta_command`

Old behavior:

```
if t == "unequip":
    char.weapon = None
    print(f" [{char.name} puts their weapon away.]")
    return True
```

New behavior:

```
if t == "unequip":
    # v2.7.6.1: putting a weapon away must not discard its instance.
    # Ammo/condition belong to the physical weapon, not the catalog entry.
    if char.weapon:
        char.weapon_instances[char.weapon.name] = char.weapon
    char.weapon = None
    print(f" [{char.name} puts their weapon away.]")
    return True
```

3.4 Replace the successful equip assignment in `_meta_command`

Find this block near the end of the equip branch:

```
char.weapon = copy.copy(w)    # their own instance; ammo is theirs
print(f" [{char.name} readies the {w.name}.]")
return True
```

Replace with:

```
# v2.7.6.1: re-ready the same physical instance. The old catalog-copy
# behavior silently restored ammunition whenever a weapon was unequipped
# and equipped again. The v2.8.0 item registry replaces this bridge.
existing = char.weapon_instances.get(name)
if existing is None:
    existing = copy.copy(w)    # first time this character has readied it
    char.weapon_instances[name] = existing
char.weapon = existing
print(f" [{char.name} readies the {existing.name}.]")
return True
```

3.5 Replace `_apply_state_delta`

Replace the whole existing `_apply_state_delta` method with this version:

```

def _apply_state_delta(self, delta: dict):
    """Apply the safe subset of a model-produced state delta.

    v2.7.6.1 — the Truth Firewall. The model may narrate and propose, but
    canonical mechanical state is engine-owned. Rejected writes are kept in
    the validation report and shown in debug mode; they never crash a turn.
    """
    report = self.state_validator.validate(
        delta,
        characters=self.characters,
        fronts=self.fronts,
        locations=self.locations,
    )

    debug = bool(self.config.get("llm", {}).get("debug", False))
    if debug and report.rejected:
        print("\n[STATE REJECTED]")
        for rejection in report.rejected:
            print(f" - {rejection.format()}")

    delta = report.delta

    for cid, changes in (delta.get("characters") or {}).items():
        char = self.characters.get(cid)
        if not char or not isinstance(changes, dict):
            continue
        old_loc = char.location
        for k, v in changes.items():
            # Belt-and-braces: even a direct internal call cannot use this
            # path to overwrite canonical mechanical state.
            if k in ENGINE_OWNED_CHARACTER_FIELDS:
                continue
            if hasattr(char, k):
                setattr(char, k, v)
        if char.location != old_loc:
            self.spatial.move_occupant(cid, old_loc, char.location)

    for k, v in (delta.get("fronts") or {}).items():
        if k in self.fronts and isinstance(v, (int, float)):
            maximum = int(self.fronts[k].get("max", int(v)))
            self.fronts[k]["clock"] = max(0, min(int(v), maximum))

    for p in (delta.get("plot_points") or []):
        if p not in self.plot_points:
            self.plot_points.append(p)

    for trans in (delta.get("scene_transitions") or []):
        for cid, dest in trans.items():
            char = self.characters.get(cid)
            if char and dest in self.locations:

```

```

        self.spatial.move_occupant(cid, char.location, dest)
        char.location = dest

players = [c for c in self.characters.values() if c.char_type == "player"]
if players and all(p.location == players[0].location for p in players):
    self.current_scene = players[0].location

```

3.6 Keeper behavior after this patch

The model can still narrate normally. It can still queue `dice_requests`. It can still add plot points and request validated movement. But it can no longer directly overwrite the campaign's mechanical truth.

4. Targeted edits: `config/system-prompt.txt`

4.1 Replace the **MOVEMENT** section

Old:

```

MOVEMENT:
- If a player's declaration says they move/enter/go somewhere, include a scene_transitions entry
  mapping their char_id to one of the location ids in EXITS. Do not invent locations.

```

New:

```

MOVEMENT:
- If a player's declaration says they move/enter/go somewhere, include a scene_transitions entry
  mapping their char_id to one of the location ids in EXITS. Do not invent locations.
- Movement must use a destination connected to the character's current location. Never write a
  character's location directly inside state_delta.characters; the engine owns location truth.

```

4.2 Add this rule before **OUTPUT FORMAT**

```

STATE AUTHORITY:
- state_delta is a proposal channel, not direct control over the campaign.
- Never assign HP, SAN, MP, Luck, characteristics, skills, inventory, weapons, ammunition,
  location, wounds, insanity, or Cthulhu Mythos. The engine calculates and owns those fields.
- In state_delta.characters you may include only narrative-safe fields such as position,
  declared_action, personal_log, or extra notes.
- Mechanical consequences belong in dice_requests, proposed_consequences, sound_events,
  validated scene_transitions, or bounded front updates. Do not write them directly onto a character.

```

4.3 Update the sample `state_delta`

Replace the sample character line:

```

"characters": {"char_id": {"hp": 8, "san": 55, "major_wound": false}},

```

With:

```

"characters": {"char_id": {"position": "behind_cover", "extra": {"demeanor": "shaken but focused"}}},

```


The rest of the sample can remain during this compatibility release.

5. Update: `src/__init__.py`

Replace the version stamp:

```
__version__ = "2.7.6.1"
```

6. Add to `test_engine.py`

Add this import near the other project imports:

```
from src.state_validator import StateDeltaValidator
```

Insert this block before the final version-stamp check:

```
print("== v2.7.6.1 field regressions: the Truth Firewall ==")
# The roadmap's Phase 0 rule: the model owns voice, not canonical truth.
# A hostile or hallucinated state_delta must not be able to rewrite HP, SAN,
# location, inventory, weapons, or any other engine-owned field.
k, _rec = _solo_keeper()
solo = k.characters["solo"]
old_hp, old_san = solo.hp, solo.san
old_location = solo.location
old_inventory = list(solo.inventory)
old_weapon = solo.weapon

k._apply_state_delta({
    "characters": {
        "solo": {
            "hp": 1,
            "san": 0,
            "location": "corbitt_house_basement",
            "inventory": ["The Necronomicon"],
            "weapon": {"name": "Sure-Win Ray", "damage": "99D6", "base_range": 999},
            "position": "behind_cover",
        }
    }
})
check("Truth Firewall blocks direct HP/SAN writes",
      solo.hp == old_hp and solo.san == old_san)
check("Truth Firewall blocks direct location/inventory/weapon writes",
      solo.location == old_location and solo.inventory == old_inventory
      and solo.weapon == old_weapon)
check("Truth Firewall preserves approved narrative fields",
      solo.position == "behind_cover")

# Front updates remain temporarily model-compatible, but are clamped to the
```

```

# scenario's configured maximum rather than trusted blindly.
k._apply_state_delta({"fronts": {"ritual": 99, "missing_front": 2}})
check("Truth Firewall clamps known front clocks", k.fronts["ritual"]["clock"] == 6)
check("Truth Firewall rejects unknown front clocks", "missing_front" not in k.fronts)

# Movement must use the actual graph. Exterior -> ground floor is valid;
# exterior -> basement is not.
k._apply_state_delta({
    "scene_transitions": [
        {"solo": "corbitt_house_ground_floor"},
        {"solo": "corbitt_house_basement"},
    ]
})
check("Truth Firewall allows connected movement",
      solo.location == "corbitt_house_ground_floor")
check("Truth Firewall blocks teleportation",
      solo.location != "corbitt_house_basement")

# Unequip/equip must not restore ammunition from the catalog. The equipped
# weapon is a persistent physical instance until the v2.8 item registry lands.
import copy as _copy
from src.charcreate import WEAPONS
shotgun = _copy.copy(WEAPONS["12-gauge shotgun"])
solo.inventory = [shotgun.name]
solo.weapon = shotgun
solo.weapon_instances[shotgun.name] = shotgun
solo.weapon.ammo = 2
k._meta_command(solo, "unequip")
k._meta_command(solo, "equip shotgun")
check("Truth Firewall preserves ammunition through unequip/equip",
      solo.weapon is shotgun and solo.weapon.ammo == 2)

# Save/load keeps the same ammunition through the v2.7.x compatibility bridge.
restored = Character.from_dict(solo.to_dict())
check("Truth Firewall weapon instances survive save/load",
      restored.weapon_instances["12-gauge Shotgun"].ammo == 2)

# Unknown top-level state_delta sections are rejected instead of silently
# becoming future corruption vectors.
validator = StateDeltaValidator()
report = validator.validate(
    {"hp_override": {"solo": 1}},
    characters=k.characters,
    fronts=k.fronts,
    locations=k.locations,
)
check("Truth Firewall rejects unknown top-level delta fields",
      report.delta == {} and len(report.rejected) == 1)

```

Update the version assertion:

```
check("version stamped", __version__ == "2.7.6.1")
```

7. New file: docs/v2.7.6.1-release-notes.md

```
# v2.7.6.1 Release Notes — “The Truth Firewall”
```

```
**Phase:** Persistent Campaign Engine Roadmap, Phase 0
```

```
**Baseline:** v2.7.6 “The Spoiler Channel”
```

```
## What changed
```

```
### The LLM no longer gets direct write access to canonical truth
```

v2.7.6 still allowed model-produced ``state_delta`` content to assign many core character fields
↳ directly. That violated the campaign-engine rule: **the engine owns truth; the model owns voice.**

This patch introduces ``src/state_validator.py``, a validation layer between the model and world state.

The model may still return:

- narration
- private narrations
- dice requests
- plot-point proposals
- bounded front-clock updates
- validated scene transitions
- sound-event proposals
- safe narrative character fields such as position, declared action, personal log, and extra notes

The model may **not** directly assign engine-owned fields such as:

- HP, SAN, MP, Luck, or characteristics
- skills or checked skills
- weapon, weapon instances, inventory, or key items
- location
- wounds, dying, unconsciousness, or insanity
- Cthulhu Mythos
- character identity fields

Rejected writes are logged in debug mode and never crash the turn.

```
### Movement is validated against the actual map
```

Scene transitions now must reference:

1. a known character,
2. a known destination, and
3. a destination connected to that character's current location.

The model can no longer teleport someone by writing a location directly into ``state_delta.characters``.

Front clocks are clamped

Model-suggested front updates must:

- reference an existing front,
- be numeric, and
- remain inside ``0..max``.

Phase 3 will replace model-driven clock changes with deterministic trigger-driven updates. This release
↪ adds the safety rail first.

Weapon ammunition persists through unequip/equip

Characters now carry persistent ``weapon_instances`` keyed by weapon display name.

Before this patch:

1. fire two shells,
2. unequip the shotgun,
3. equip it again,
4. receive a fresh catalog copy with full ammunition.

Now re-equipping retrieves the same weapon instance. Ammunition and malfunction state survive unequip/
↪ equip and save/load. The full item/object registry arrives in v2.8.0; this is the compatibility
↪ bridge.

Prompt serialization includes carried gear

``Character.to_active_format()`` now includes inventory alongside the equipped weapon, reducing the
↪ chance that the model narrates gear the character does not actually possess.

Files changed

- ``src/__init__.py`` — version stamp becomes ``2.7.6.1``
- ``src/state_validator.py`` — new Truth Firewall
- ``src/character.py`` — persistent weapon instances and inventory prompt visibility
- ``src/keeper.py`` — validator integration, rejected-write logging, persistent equip flow
- ``config/system-prompt.txt`` — model output contract tightened
- ``test_engine.py`` — Phase 0 regression coverage
- ``docs/v2.7.6.1-release-notes.md`` — this file

Regression coverage

New tests prove that:

- the LLM cannot directly set HP, SAN, location, inventory, or weapons;
- unapproved character fields are rejected;
- unknown characters cannot receive state changes;

- front clocks clamp to their configured maximum;
- scene transitions must follow the location graph;
- plot points and sound events are sanitized;
- unknown top-level ``state_delta`` fields are rejected;
- weapon instances seed from equipped weapons;
- ammunition persists through unequip/equip;
- ammunition persists through save/load.

Compatibility

Existing v2.7.x saves migrate on load:

- equipped weapons seed ``weapon_instances``;
- empty inventories still seed from equipped weapons;
- old saves without ``weapon_instances`` load normally.

The legacy ``state_delta`` shape is still accepted, but only after validation. This keeps current provider
↔ behavior compatible while closing the direct-mutation hole.

Next phase

****v2.8.0 — Canonical Items and Objects****

- item templates and item instances
- persistent non-weapon gear
- keys, documents, containers, consumables, and clue items
- room object registry
- take/drop/give/reload/open commands
- item-aware combat and object state

8. Standalone regression tests used for this progress pack

These tests were run directly against the new validator and character model.

tests/test_state_validator.py

```
from types import SimpleNamespace

from src.state_validator import StateDeltaValidator

def fixture_world():
    characters = {
        "eleanor": SimpleNamespace(location="nave"),
        "samuel": SimpleNamespace(location="vestry"),
    }
    locations = {
        "nave": SimpleNamespace(connections={"vestry": {}, "crypt": {}}),
```

```

        "vestry": SimpleNamespace(connections={"nave": {}}),
        "crypt": SimpleNamespace(connections={"nave": {}}),
    }
    fronts = {
        "choir_below": {"clock": 2, "max": 6},
    }
    return characters, fronts, locations

def validate(delta):
    characters, fronts, locations = fixture_world()
    return StateDeltaValidator().validate(
        delta, characters=characters, fronts=fronts, locations=locations
    )

def test_blocks_engine_owned_character_fields():
    report = validate({
        "characters": {
            "eleanor": {
                "hp": 1,
                "san": 0,
                "location": "crypt",
                "inventory": ["Necronomicon"],
                "position": "behind_cover",
            }
        }
    })
    assert report.delta == {"characters": {"eleanor": {"position": "behind_cover"}}}
    assert len(report.rejected) == 4
    assert all("engine-owned" in r.reason for r in report.rejected)

def test_rejects_unknown_character_and_unapproved_field():
    report = validate({
        "characters": {
            "nobody": {"hp": 3},
            "eleanor": {"favorite_color": "green"},
        }
    })
    assert report.delta == {}
    assert len(report.rejected) == 2

def test_fronts_are_known_numeric_and_clamped():
    report = validate({"fronts": {"choir_below": 99, "missing": 2}})
    assert report.delta == {"fronts": {"choir_below": 6}}
    assert len(report.rejected) == 1

def test_scene_transitions_must_be_connected():

```

```

report = validate({
    "scene_transitions": [
        {"eleanor": "crypt"},
        {"samuel": "crypt"},
    ]
})
assert report.delta == {"scene_transitions": [{"eleanor": "crypt"}]}
assert len(report.rejected) == 1
assert "not connected" in report.rejected[0].reason

def test_plot_points_and_proposals_are_sanitized():
    report = validate({
        "plot_points": ["ledger_found", "", 42],
        "sound_events": [
            {"location": "nave", "noise": 4, "heard_by": ["eleanor", "nobody"]},
            {"location": "missing", "noise": 4, "heard_by": []},
        ],
        "proposed_facts": [{"subject": "door", "predicate": "broken"}],
    })
    assert report.delta["plot_points"] == ["ledger_found"]
    assert report.delta["sound_events"] == [
        {"location": "nave", "noise": 4, "heard_by": ["eleanor"]}
    ]
    assert report.delta["proposed_facts"] == [{"subject": "door", "predicate": "broken"}]
    assert len(report.rejected) == 3

def test_unknown_top_level_field_is_rejected():
    report = validate({"hp_override": {"eleanor": 1}})
    assert report.delta == {}
    assert len(report.rejected) == 1

```

tests/test_character.py

```

from src.character import Character, Weapon

def test_equipped_weapon_seeds_persistent_instance_and_inventory():
    gun = Weapon(name="12-gauge Shotgun", damage="2D6", base_range=50, ammo=5, is_shotgun=True)
    char = Character(id="eleanor", name="Eleanor", char_type="player", weapon=gun)
    assert char.inventory == ["12-gauge Shotgun"]
    assert char.weapon_instances["12-gauge Shotgun"] is gun

def test_weapon_instance_ammo_round_trips_save_format():
    gun = Weapon(name="12-gauge Shotgun", damage="2D6", base_range=50, ammo=3, is_shotgun=True)
    char = Character(id="eleanor", name="Eleanor", char_type="player", weapon=gun)
    restored = Character.from_dict(char.to_dict())
    assert restored.weapon_instances["12-gauge Shotgun"].ammo == 3
    assert restored.weapon.ammo == 3

```

```
def test_reequip_can_reuse_existing_instance():
    gun = Weapon(name=".32 Revolver", damage="1D8", base_range=15, ammo=6, is_short_barrel=True)
    char = Character(id="eleanor", name="Eleanor", char_type="player", weapon=gun)
    char.weapon.ammo = 2
    char.weapon_instances[char.weapon.name] = char.weapon
    char.weapon = None
    char.weapon = char.weapon_instances[".32 Revolver"]
    assert char.weapon.ammo == 2
```

9. Test results from this implementation pass

The standalone Phase 0 tests passed:

```
PASS test_blocks_engine_owned_character_fields
PASS test_fronts_are_known_numeric_and_clamped
PASS test_plot_points_and_proposals_are_sanitized
PASS test_rejects_unknown_character_and_unapproved_field
PASS test_scene_transitions_must_be_connected
PASS test_unknown_top_level_field_is_rejected
PASS test_equipped_weapon_seeds_persistent_instance_and_inventory
PASS test_reequip_can_reuse_existing_instance
PASS test_weapon_instance_ammo_round_trips_save_format
```

The full project suite could not be executed here because the upload contained only the complete-source PDF rather than the editable source tree. Once these blocks are merged into the actual project, run:

```
python test_engine.py
python test_charcreate.py
python -m src.main --mock
```

10. Next implementation target

Proceed to **v2.8.0 — Canonical Items and Objects** after v2.7.6.1 passes the full offline suite in the editable project.

Recommended first v2.8.0 tasks:

1. Create `src/items.py` with `ItemTemplate` and `ItemInstance`.
2. Add `World.items` and room item references.
3. Migrate `Character.inventory` from display names to item IDs.
4. Move `weapon_instances` into the item registry compatibility layer.
5. Add persistent `take`, `drop`, `give`, `reload`, and `open` commands.
6. Add room object records for doors, containers, clue objects, and destructible scenery.
7. Add save/load migration from v2.7.6.1.